```
➤          return RecordResult.new(false, nil, message)
➤        end

         DB[:expenses].insert(expense)
         id = DB[:expenses].max(:id)
         RecordResult.new(true, id, nil)
       end

       def expenses_on(date)
       end
     end
   end
```

Rerun your specs; they should all pass. Well done! You've implemented saving a single expense at all layers of the app, from top to bottom.

```
$ bundle exec rspec spec/integration/app/ledger_spec.rb
« truncated »

Finished in 0.01205 seconds (files took 0.15597 seconds to load)
2 examples, 0 failures

Randomized with seed 38450
```

Now that you've got the hang of the process, and have put in all the data and routing infrastructure, the next bit of Ledger behavior will be much easier to implement.

Querying Expenses

Let's turn our attention to the final piece of the puzzle: querying expenses back from the database. First, you'll need a failing spec that records a few expenses into the ledger, with some of the expenses on the same date. Querying for that date should return only the matching expenses.

Add the following specs inside the RSpec.describe block in spec/integration/app/ledger_spec.rb:

06-integration-specs/09/expense_tracker/spec/integration/app/ledger_spec.rb

```
describe '#expenses_on' do
  it 'returns all expenses for the provided date' do
    result_1 = ledger.record(expense.merge('date' => '2017-06-10'))
    result_2 = ledger.record(expense.merge('date' => '2017-06-10'))
    result_3 = ledger.record(expense.merge('date' => '2017-06-11'))

    expect(ledger.expenses_on('2017-06-10')).to contain_exactly(
      a_hash_including(id: result_1.expense_id),
      a_hash_including(id: result_2.expense_id)
    )
  end
```